

AGENTIC SYSTEMS | M&A RESEARCH | INSTITUTIONAL ADAPTATION

DYNAMIC AGENTIC ECOSYSTEMS IN M&A RESEARCH

From Approved Specialists to Adaptive, Auditable Research Institutions

A pedagogical monograph aligned to Notebook 05, explaining how constellations discover capability, negotiate bounded work, share revision-aware knowledge, form temporary providers, reorganize under disturbance, and preserve an institutional audit trail.

PAPER PURPOSE

Notebook 05 is the capstone of a cumulative tutorial on agentic systems. The notebook does not ask a single language model to perform an entire acquisition analysis. Instead, it represents an M&A research institution as a governed ecosystem of specialized constellations that can discover one another, negotiate bounded work agreements, reserve scarce specialist capacity, publish evidence with provenance, reuse knowledge only when it remains valid, create temporary providers from approved primitives, reorganize when the environment changes, and preserve a complete record of what happened. The paper follows that executable architecture unit by unit and explains why each boundary matters for adaptive, auditable financial systems.

Alejandro Reynoso

AI for Financial Practitioners | Dynamic Agentic Systems

September 2026

Notebook 05 / Topic 766

Executive Overview

THE NOTEBOOK IN ONE VIEW

Notebook 05 is the capstone of a cumulative tutorial on agentic systems. Its purpose is not to demonstrate that a single large language model can perform merger-and-acquisition analysis. Rather, it shows how a financial-research institution can be represented as a governed ecosystem of constellations that discover one another, negotiate bounded work agreements, reserve scarce specialist capacity, publish evidence with provenance, reuse knowledge only when it remains valid, create temporary providers from approved primitives, reorganize when the environment changes, and preserve a complete institutional record of what happened. The notebook is intentionally pedagogical: it runs in one Python process, uses a synthetic M&A world, supports a deterministic simulation mode, and exposes a bounded LIVE mode in which `gpt-5-nano` may perform specialist interpretation, synthesis, and temporary-provider selection under strict schemas and request limits. This image-free paper follows the notebook exactly, unit by unit. The aim is to explain not only what each block of code does, but why the architectural boundary it introduces matters for building adaptive, auditable, financially serious agentic systems.

DESIGN PRINCIPLE

The central architectural move is to separate **approved capability** from **current organization**. Specialists and tools are governed primitives; the directory, contracts, alliances, knowledge state, and temporary providers are organizational variables. The ecosystem becomes adaptive by changing relationships among approved capabilities rather than by allowing unrestricted self-modification.

Introduction: From a Team of Agents to an Adaptive Institutional Ecosystem

Notebook 05 begins from a simple but consequential observation: a constellation can itself become a participant in a larger system. In the earlier stages of the tutorial, the basic unit of agency was progressively enlarged. First there were individual agents equipped with deterministic tools and reusable skills. Then agents were grouped into constellations that could coordinate specialized work. Feedback loops added memory and the ability to revise future behavior. Notebook 04 then allowed the system to reconfigure the internal composition of a constellation by examining a mandate, discovering the capabilities required, selecting approved specialists from a registry, and validating the resulting organization before allowing it to act. Notebook 05 takes the next architectural step. Instead of asking how one constellation adapts internally, it asks how several constellations can coexist, retain separate identities, exchange work through explicit agreements, and reorganize their relationships when the problem changes.

The domain is a synthetic merger-and-acquisition research laboratory. The learner should resist the temptation to think of the notebook as an automated deal engine. It does not negotiate a real acquisition, commit capital, issue a legal opinion, or execute a transaction. Its institutional object is much narrower and much more useful pedagogically: the research organization that supports a transaction team. A finance desk may evaluate growth, valuation multiples, leverage, and enterprise value. A strategy desk may assess strategic fit, geography, business-model characteristics, and cross-border issues. An intelligence desk may examine reports, analyst opinions, and rumors while retaining document identifiers and source reliability. Additional approved specialists exist for integration work and as backup resources. Notebook 05 asks how such organizational units can discover each other, contract with each other, share research, and adapt when the mission changes.

The distinction between an agent and an organization is central. The notebook does not simply give every specialist access to every other specialist and call the result an ecosystem. It creates a directory of desk constellations. Each desk has members, a coordinating identity, an active or inactive state, and a flag indicating whether it is temporary. The mission coordinator does not directly manipulate every specialist action. Instead, it asks which desks can currently provide a capability. A provider is discoverable only if the desk is active, the relevant specialist is available, the specialist owns the required capability, and that specialist is not already reserved by another contract. Capability therefore becomes a dynamic organizational property rather than a static label in a registry.

This is where the architecture becomes institutionally interesting. The global registry and the organizational directory are deliberately different objects. The registry answers a personnel-and-capability question: which approved specialists exist, what can they do, what do they cost in synthetic teaching units, and are they available? The directory answers a structural question: which specialists currently belong to which active organizational providers? An integration analyst can exist in the registry without an integration desk existing in the active ecosystem. Likewise, a backup financial analyst can exist as an approved reserve primitive without being part of the finance desk. This separation creates the possibility of organizational formation. If a mission requires a capability that no active desk currently provides, the ecosystem may create a temporary provider from an approved but unallocated specialist. The system is not inventing new executable intelligence. It is reorganizing already-governed intelligence into a new institutional configuration.

The notebook also makes interaction explicit through a protocol. Requests for proposals, bids, counteroffers, acceptances, rejections, deliveries, cancellations, and knowledge-sharing events are represented as messages with senders, recipients, task identifiers, revisions, and payloads. That design matters because it turns coordination into inspectable state. A conventional Python program could simply call a function on the chosen analyst and move on. Notebook 05 instead forces the learner to see the social structure of the computation. A finance contribution is not merely a return value. It is a deliverable produced under a reserved contract, associated with a specific provider, specialist, capability, candidate scope, and environment revision.

Negotiation is intentionally bounded. Providers quote a synthetic price based on the specialist's declared cost. The mission coordinator makes one counteroffer. The provider accepts only if the offer is at least the specialist's reservation price and the specialist remains available. The notebook does not attempt to simulate a full market for research services or an open-ended bargaining dialogue between language models. This restriction is a feature, not a weakness. It allows the learner to observe the difference between discovery, negotiation, agreement, reservation, delivery, and release without hiding those transitions inside an unconstrained conversation. The resulting protocol echoes classic distributed problem-solving ideas such as the Contract Net Protocol, where work is allocated through explicit requests and offers rather than by one permanently centralized scheduler (Smith, 1980).

Memory is equally governed. Reports delivered by specialists are stored on a knowledge board together with the originating provider, task identifier, buyer, candidate scope, capability, and environment revision. A later mission may reuse such knowledge, but only if the lookup key matches all relevant dimensions. A report about a different buyer is not automatically reusable. A report comparing a different candidate set is not automatically reusable. Most importantly, a report generated under an earlier environment revision is retained for audit but is no longer considered fresh evidence. Notebook 05 therefore distinguishes institutional memory from institutional truth. The system may remember everything while still refusing to treat old knowledge as current.

The role of revision is one of the most important pedagogical devices in the notebook. Environmental disturbances are modeled explicitly: the mandate may expand, a provider may withdraw, or a target may become unavailable. Before a disturbance changes the revision, outstanding reserved work is cancelled. The event is recorded, the revision is incremented, and future knowledge lookups operate under the new state. This makes adaptation visible. The organization does not merely continue executing a static workflow against changed assumptions. It invalidates unfinished commitments, recognizes that previous evidence may be stale, and reconstructs the mission under the new conditions.

The notebook supports two execution modes. In **SIMULATION**, there are no model API calls. Specialist choices and synthesis use deterministic teaching fallbacks. In **LIVE**, the notebook loads **gpt-5-nano** through the OpenAI Responses API, constrains outputs with JSON schemas, limits calls and response tokens, caches identical requests, and records token usage. Crucially, the language model operates inside the institutional architecture rather than defining it. The code still determines which tools an agent may use, which specialists are available, whether a contract is valid, whether a delivery is stale, which evidence identifiers are permissible, and whether a final mission has complete capability coverage. The model may interpret, select, or synthesize; the architecture decides what is permissible.

This distinction is consistent with a broad direction in modern agentic-system research. ReAct separates reasoning from environment-facing actions (Yao et al., 2023). Reflexion shows how retained feedback can influence later behavior without retraining the underlying model (Shinn et al., 2023). AutoGen, MetaGPT, and AgentVerse explore structured multi-agent interaction, role specialization, workflow coordination, and changing group composition (Wu et al., 2024; Hong et al., 2024; Chen et al., 2024). Notebook 05 translates these themes into a deliberately transparent financial-research laboratory in which roles, permissions, resource constraints, communication, memory, and organizational lifecycle can be inspected directly. The executable notebook sequence and the papers used to frame its research foundations are maintained in the public *AI Finance: From Agents to Ecosystems* notebook repository (Reynoso, 2026); this monograph is intended to be read alongside that repository corpus.

The notebook's deepest lesson is therefore organizational rather than model-centric. Intelligence is distributed across approved capabilities. Reliability is supplied by contracts, revisions, permissions, schemas, provenance, validation, and cleanup rules. Adaptation occurs because the organization can form, reuse, dissolve, and reform relationships as the mission changes. The model contributes reasoning; the system around the model creates institutional discipline.

Code Unit 1: Restore the Common M&A World and Establish Execution Controls

The first code unit is a continuity layer. It restores the same synthetic M&A world used earlier in the tutorial and establishes the execution environment in which the ecosystem will operate. This may look like ordinary setup code, but it performs several functions that are essential if later organizational behavior is to be interpreted correctly.

The notebook points to four student-facing data files: `companies.csv`, `financials.csv`, `strategic_profiles.csv`, and `documents.csv`. The code verifies that the three company-level tables each contain 500 unique companies and share the same company identifiers. It then merges those tables into a single universe and verifies that the document table contains 1,500 unique documents, exactly three per company. The notebook also checks that required numerical fields such as EV/EBITDA, net debt/EBITDA, revenue growth, and enterprise value are finite. These are not cosmetic assertions. They establish the invariants on which later screening and specialist tools depend.

The data files are then fingerprinted with SHA-256 hashes. This is the first important provenance decision in NB05. Later, when Notebook 05 imports the capability handoff from Notebook 04, the source fingerprints must match. The ecosystem is therefore not permitted to claim continuity with a prior organizational state while silently operating on a different dataset. That linkage becomes part of the final manifest in Unit 10.

The code next initializes a fresh global state with revision zero, no blocked companies, and an empty event log. This is deliberately different from importing unfinished scenario state from NB04. Notebook 05 inherits capability definitions and reviewed organizational primitives from Notebook 04, but it starts a new ecosystem runtime. This distinction is pedagogically important: definitions may be inherited; commitments should not be. An uncompleted reservation, stale alliance, or partially executed mission from another notebook should not leak into the new experiment.

Execution mode is made explicit. `MODE` is either `SIMULATION` or `LIVE`, and the configured model is `gpt-5-nano`. In simulation mode, no API client is created and no model calls occur. In live mode, the notebook mounts Google Drive when needed, installs the OpenAI Python package if it is absent, retrieves `OPENAI_API_KEY` from Colab Secrets, and creates a client with a finite timeout and no automatic retries. The explicit request budget and output-token ceiling further bound the experiment. This is a good example of a design principle that will recur throughout the notebook: scarce resources should be represented as explicit state rather than hidden assumptions.

The separation between the shared dataset and the output directory is equally significant. Notebook 05 never rewrites the four source CSV files. Results go to a sibling `OUTPUTS` directory. The teacher-only key used to generate or validate the synthetic world is not loaded. The learner therefore works with the same information boundary as the student-facing system.

A useful way to understand Unit 1 is as a constitutional layer. Before the ecosystem is allowed to negotiate, reorganize, or reuse memory, it establishes what world it is in, what evidence is available, which mode is active, how many model calls are permitted, and where the audit trail will be written. Adaptive behavior becomes meaningful only because these background conditions are stable.

```
MODE = os.environ.get("MA_MODE", "SIMULATION")
MODEL = "gpt-5-nano"
STATE = {"revision": 0, "blocked": [], "events": []}
USAGE = {"calls": 0, "input_tokens": 0,
         "output_tokens": 0, "cache_hits": 0}
MAX_CALLS, MAX_OUTPUT_TOKENS = 80, 2400
```

Pedagogical interpretation. The ecosystem is adaptive only after the environment has been fixed and verified. Reproducibility begins with data identity, explicit runtime state, explicit execution mode, and bounded access to external model resources.

Code Unit 2: Reconstruct Approved Primitives Inside a Fresh Runtime

Code Unit 2 reconstructs the analytical primitives that the ecosystem is allowed to organize. The notebook does not assume that Notebook 04 is still alive in memory. Instead, it rebuilds the skills, deterministic tools, permission-aware dispatcher, model interface, and specialist execution contract inside the new runtime. This is a substantial pedagogical point: the ecosystem depends on stable interfaces, not on fragile in-memory continuity.

Five analytical capabilities are defined. Finance compares growth, EV/EBITDA, net leverage, and enterprise value while explicitly avoiding financing commitments. Strategy compares sector, geography, strengths, weaknesses, and acquisition appetite while acknowledging integration uncertainty. Intelligence separates reports, analyst opinions, and rumors, retaining document identifiers and reliability. Cross-border review flags country differences and synthetic regulatory sensitivity without pretending to provide legal clearance. Integration review compares business models and complexity while explicitly refusing to infer a synergy estimate from the limited evidence.

Each capability maps to exactly one deterministic tool. The registry then associates approved agents with one capability, one teaching cost, an availability flag, and the corresponding allowed tool. The key idea is that an agent is not authorized merely because a Python function exists. `call_tool()` checks whether the agent is known and available and whether the requested tool appears in that agent's allowlist. A specialist therefore operates inside a permission boundary.

The deterministic tools produce structured evidence, but the specialist layer is responsible for interpretation. The function `specialist()` calls the authorized tool for every candidate in the mandate, builds a schema for the expected result, and then asks either the simulation policy or the live language model to choose one candidate, summarize the reasoning, and cite only document identifiers actually supplied by the tool. The result is validated again: the chosen company must be inside the candidate set, and cited document IDs must be a subset of the evidence supplied to the specialist.

The model interface, `ask_json()`, deserves close attention. Every request is hashed together with the configured model, execution mode, current revision, task, and schema. If the hash is present in the cache, the notebook returns the cached result and increments the cache-hit count. Otherwise, simulation mode returns the declared deterministic fallback. Live mode reserves one of the finite model requests before transmission, calls the Responses API with low reasoning effort and a strict JSON schema, records input and output tokens, rejects incomplete or refused responses, and verifies the result shape before accepting it. This means that “using an LLM” is not treated as a mystical transition from deterministic to intelligent behavior. It is a bounded service call with a contract, a budget, a cache, an audit context, and validation.

This unit also clarifies the relationship between intelligence and architecture. The specialist does not decide which desk it belongs to, whether a contract should be signed, whether another desk should join the mission, or whether a previous report can be reused. Those responsibilities are deliberately outside the specialist function. The specialist's institutional role is narrow: use permitted evidence and an approved analytical method to produce one capability-specific contribution.

The benefit of this separation becomes obvious in later units. A financial analyst can move from one provider to another without rewriting the finance tool. A backup financial analyst can substitute for the primary analyst because both satisfy the same capability contract. A temporary integration desk can be formed because the integration analyst already exists as an approved primitive. Stable capability contracts therefore make organizational reconfiguration possible without redefining analytical meaning.

Code Unit 3: Import the NB04 Handoff and Define the Constellation Directory

Unit 3 is the explicit bridge between a self-adapting constellation and a dynamic ecosystem. Notebook 05 does not merely trust that its local registry resembles the one used in Notebook 04. It reads the pointer file `NB04_latest.json`, locates the handoff artifact, verifies the handoff checksum, validates the schema version, confirms that the dataset fingerprints match the current data, and finally checks that the imported declarative registry and skill definitions are identical to the reviewed executable definitions in NB05. Only after these checks succeed does the notebook accept the handoff.

This sequence is a practical illustration of provenance across notebooks. The ecosystem inherits capability definitions from a prior stage of the tutorial, but it does so through an explicit, verifiable artifact. In a larger institutional architecture, the same principle would apply to model registries, policy versions, approved tool catalogs, and risk sign-offs. A later system should be able to say not only which capabilities it is using, but which reviewed version of those capabilities it inherited.

The new concept introduced in Unit 3 is the directory. Three active desk constellations are defined. The finance desk contains the primary financial analyst. The strategy desk contains both the strategy analyst and cross-border analyst. The intelligence desk contains the intelligence analyst. Each desk has a coordinator identity. All three begin active and non-temporary. The integration analyst and backup financial analyst remain in the global registry but are not assigned to an active desk.

The distinction between **REGISTRY** and **DIRECTORY** is the architectural core of the unit. The registry describes approved primitives. The directory describes the current organization. An approved specialist may exist without being organizationally available as a provider. This is precisely what later enables formation: the ecosystem can discover a gap at the directory level and fill it from the registry level without inventing a new skill.

The unit also defines the mandate-to-capability mapping. Every mission requires finance, strategy, and intelligence. A cross-border mandate additionally requires cross-border analysis. A mandate with integration review additionally requires integration. The mission is therefore translated into an explicit capability set before organizational work begins.

Candidate screening is also reconstructed here. The function `make_shortlist()` removes the buyer, excludes blocked companies, applies EV/EBITDA and leverage thresholds, imposes domestic or foreign-country scope depending on the mandate, computes a transparent screening score, and returns the top three eligible candidates. Importantly, the function refuses to silently relax the scope if fewer than two candidates remain. A constrained system should fail visibly when its mandate becomes infeasible rather than quietly changing the mandate.

Finally, `integrate()` requires complete specialist coverage and current revision alignment before a decision can be synthesized. It verifies that all required capabilities are represented, rejects stale reports, re-screens the candidate universe to detect stale candidate lists, counts specialist preferences, selects the candidate with the most votes, and asks the deal lead to explain evidence, disagreements, and open diligence questions. The final status is explicitly `research_only`. The ecosystem coordinates research; it does not approve or execute an acquisition.

Pedagogical interpretation. The registry is a catalog of approved capability; the directory is the currently active institution. Dynamic ecosystems become possible when the system can change the second without corrupting the first.

Code Unit 4: Make Communication, Discovery, and Knowledge Freshness Explicit

Code Unit 4 introduces three mechanisms that make the ecosystem observable: a message bus, capability discovery, and revision-aware knowledge lookup. These mechanisms are deliberately local Python structures rather than external distributed infrastructure. The simplification lets the learner inspect the protocol directly before introducing networking, asynchronous delivery, authentication services, or distributed storage.

The message bus accepts only eight message kinds: RFP, BID, COUNTER, ACCEPT, REJECT, DELIVER, CANCEL, and SHARE. Every message receives a unique message identifier and records the sender, recipient, task identifier, current revision, and a copied payload. Unknown message types are rejected. Unknown participants are rejected. The set of known participants includes active directory entries plus the mission coordinator and knowledge board. Communication is therefore not free-form text. It is a typed protocol with an explicit address space.

The function `available_agents()` introduces dynamic availability. A desk can supply a capability only if the desk is active, a member possesses the requested capability, that member is marked available in the registry, and the member is not currently present in `BUSY_AGENTS`. Discovery is therefore recomputed from current organizational state. It is not equivalent to asking which desk historically contained a finance analyst.

This matters because later disturbances change the meaning of discovery. When the finance desk withdraws, the desk becomes inactive and its members are marked unavailable. Finance discovery must then fail even though the registry still records that a financial analyst once existed. The ecosystem subsequently forms a replacement provider around the approved backup analyst. Discovery is thus a statement about present organizational capacity.

The third mechanism, `fresh_knowledge()`, is one of the notebook's most important governance devices. The key is constructed from four elements: capability, buyer identifier, candidate list, and current revision. Reuse is allowed only when all four match. This prevents several subtle errors. A strategic report about one buyer cannot automatically be reused for another buyer. A report comparing three targets cannot automatically satisfy a later mandate with a different candidate list. Most importantly, a report generated before the environment changed cannot silently become current evidence merely because its text still looks relevant.

The notebook deliberately preserves old knowledge. Stale reports remain on the board and remain inspectable for audit. They simply fail the current lookup. This is a mature conception of memory. Institutional memory should be durable, but decision validity should be conditional. The system remembers what it knew; it does not pretend that what it knew is still sufficient.

The initial printed discovery results make the idea concrete. Finance has an active provider. Integration does not, even though an approved integration analyst exists in the registry. That contrast prepares the learner for temporary provider formation in Unit 7. The capability gap is organizational, not computational.

Code Unit 5: Negotiate Bounded Contracts and Reserve Specialist Capacity

The fifth unit converts capability discovery into a governed commitment. Until this point, a mission coordinator may know that a desk can provide finance or strategy, but no work has been promised. `negotiate()` creates the protocol that turns a discoverable capability into a reserved contract.

A unique task identifier is created first. The coordinator discovers all current providers of the required capability. If none exist, negotiation cannot proceed. For each provider, the mission coordinator sends an RFP containing the capability, buyer, candidate scope, and remaining budget. The provider selects its cheapest currently available specialist for that capability. The specialist's registry cost becomes the reservation floor, and the provider quotes `floor + 2`. A BID message records that offer.

The coordinator chooses the lowest quote, breaking ties by desk name. It then makes exactly one counteroffer: one unit below the winning quote, capped by the remaining mission budget. This rule is deliberately deterministic. The notebook does not ask two language models to bargain until they converge. That would introduce ambiguity about termination, cost, and reproducibility. Instead, the protocol demonstrates the institutional meaning of an offer and a counteroffer while keeping the negotiation boundary inspectable.

The provider accepts only if the counteroffer is at least its floor. Otherwise it sends a REJECT message and the negotiation fails. Even after the economic rule is satisfied, capacity is checked again to make sure the winning specialist has not become unavailable. Losing providers receive explicit rejection messages. The winning provider sends ACCEPT, and the contract is written with the agreed price, provider desk, specialist, capability, revision, mandate, and status `reserved`.

Reservation changes system state. The specialist is added to `BUSY_AGENTS`, which means another request can no longer discover that specialist as available. The task-to-capability mapping is also recorded. This is a crucial transition. A contract is not simply a narrative record of agreement; it changes future organizational possibilities.

The synthetic prices must be interpreted correctly. They are teaching resource units, not API token costs, investment-banking fees, or estimates of labor compensation. Their role is to make scarcity observable. The mission has a finite team budget. Specialists have different declared costs. The organization must therefore allocate work inside a bounded resource envelope. This makes failures meaningful. In Unit 9, an insufficient-budget mission is expected to fail without leaving anyone reserved.

The negotiation protocol is a simplified descendant of classic distributed task-allocation ideas. The Contract Net Protocol similarly distinguishes announcement, bidding, award, and execution in distributed problem solving (Smith, 1980). Notebook 05 does not claim to reproduce a market. It uses the same structural insight: distributed capability becomes governable when requests and commitments are explicit objects.

Code Unit 6: Deliver Contracted Evidence and Publish Provenance-Aware Knowledge

Code Unit 6 separates agreement from performance. A reserved contract means that a provider has committed capacity; it does not mean that valid analytical work has been delivered. The function `deliver()` enforces the distinction.

Delivery begins by checking contract state. Only a contract with status `reserved` may be delivered. A second attempt to deliver an already completed contract is rejected as a duplicate. The contract revision must equal the current global revision, and the provider desk must still be active. If either condition fails, the contract is cancelled, the specialist reservation is released, and the caller is told that the work must be replanned. This is a direct safeguard against stale execution.

If the contract remains valid, the assigned specialist performs its capability-specific analysis using the tool and model interface defined in Unit 2. The delivered report must advertise the same capability that the contract requested. A `DELIVER` message then records the provider, agent, capability, and revision. This is followed by publication to the knowledge board.

The knowledge board entry contains more than the report. It records the provider and task identifier that produced the evidence. A `SHARE` message records the publication event and retains the document identifiers associated with the report. The result is a provenance chain from mandate to task, from task to contract, from contract to specialist, from specialist to tool evidence, from evidence to report, and from report to the knowledge board.

The failure path is equally important. If specialist execution raises an exception or produces an invalid capability, the contract is cancelled and the reservation is released. A `finally` block discards the specialist from `BUSY_AGENTS` regardless of whether the delivery succeeds or fails. This is a small piece of code with a large institutional meaning: capacity cleanup is part of correctness.

The notebook's approach to provenance is particularly relevant in finance. A final research memo that says "Target C137 is preferred" is much less useful than a chain showing which specialists contributed, which evidence each specialist used, what revision of the environment was current, which provider delivered the work, and whether the contribution was new or reused. The architecture makes such reconstruction possible.

It also prevents a common failure mode in agentic applications: treating memory as an anonymous text heap. Here, a report retains an origin. A later mission that reuses the report can still identify the task and provider that created it. Reuse therefore reduces new work without erasing accountability.

The notebook's `LIVE` mode adds another provenance dimension. Model requests and token usage are counted, and exact repeated requests within the same revision can be served from cache. A repeated answer is therefore distinguishable from a fresh model call. This is not merely a cost optimization; it is part of the institutional record.

Pedagogical interpretation. A contract creates an obligation; delivery creates evidence. The architecture treats them as separate states because the institution must know whether work was merely promised, successfully completed, cancelled, or rendered stale by a changed environment.

Code Unit 7: Form Temporary Providers and Execute a Mission Alliance

Unit 7 is where the separate mechanisms become an operating ecosystem. The function `run_ecosystem()` combines shortlist construction, capability requirements, knowledge lookup, provider discovery, temporary formation, negotiation, reservation, delivery, synthesis, dissolution, and cleanup into one mission lifecycle.

The mission begins with a copy of the mandate and a freshly computed shortlist. An alliance record is created with status **forming**. It tracks participating providers, contracts, revision, spending, reused reports, and temporary desks created for the mission. The notebook then processes every required capability.

For each capability, the knowledge board is checked first. If compatible knowledge exists under the same capability, buyer, candidate list, and revision, the report is reused. A SHARE message records that reuse, including the original provider. No new contract cost is incurred. This is why the second baseline mission can satisfy the same requirements with fewer new contracts: institutional memory replaces redundant work when the scope and state are unchanged.

If no compatible knowledge exists, the ecosystem attempts discovery. When discovery fails, `form_provider()` searches the global registry for an approved, available, unallocated specialist with the missing capability. The candidate cannot already belong to an active desk and cannot be busy. In simulation mode, the lowest-cost eligible agent is chosen deterministically. In LIVE mode, `gpt-5-nano` may select among the permitted candidate identities using a constrained schema. The chosen specialist is then placed inside a temporary desk with a coordinator identity.

This is one of the most important architectural boundaries in the notebook. Temporary organizational formation does not create new code. It creates a new organizational wrapper around an approved primitive. The integration analyst, for example, already exists and already has an approved tool. The ecosystem simply had no active integration provider. Formation makes that capability discoverable without changing what the analyst is allowed to do.

The alliance reserves all required contracts before executing any deliveries. This design makes coverage and capacity conflicts visible before synthesis. Once all reservations succeed, the alliance becomes active. The contracts are delivered, the complete report set is integrated, and the alliance is marked completed. If any part fails, outstanding contracts are cancelled and the alliance is marked failed.

The **finally** block then dissolves the alliance regardless of outcome. Temporary providers are retired when their members no longer hold reservations. The historical alliance remains in the record with its outcome. This lifecycle is a powerful pedagogical representation of a deal team: organizations can assemble around a mission, contribute bounded work, and then dissolve without erasing the record of their participation.

The notebook runs two missions immediately. The first is a baseline. The second repeats the same mandate at the same revision. The second mission is expected to reuse three reports from the knowledge board. This small experiment demonstrates that memory is not merely stored; it changes future organizational demand.

Code Unit 8: Apply Environmental Disturbances and Observe Reorganization

The eighth unit is the clearest demonstration of adaptation in the notebook. Up to this point, the ecosystem has shown that it can discover providers, negotiate work, reuse knowledge, and create a temporary provider when a capability is missing. Unit 8 changes the environment repeatedly and observes whether the institution can reorganize without violating its own rules.

The function `ecosystem_event()` begins with a precaution that is easy to overlook: all outstanding reserved work is cancelled before the environment revision changes. This prevents an old commitment from being delivered into a new state. Only after cleanup does the event modify the organization or candidate universe. The revision is then incremented, the event is appended to state history, and the audit log records the disturbance.

The first disturbance expands the mandate. Cross-border analysis and integration review become required. The strategy desk already contains the cross-border analyst, so one new requirement can be satisfied by an existing provider. Integration is different. The integration analyst exists in the approved registry but no active desk provides the capability. The ecosystem therefore creates a temporary integration provider, negotiates the work, receives the report, and includes that provider in the mission alliance. This is adaptation to a capability gap.

The second disturbance is organizational rather than analytical. The finance desk withdraws. The desk becomes inactive, and all its members are marked unavailable. Finance is still required by the mandate, but the former provider can no longer satisfy discovery. The backup financial analyst remains in the approved registry. The ecosystem therefore forms a replacement provider around the backup. This is adaptation to an availability gap.

The distinction between those two cases is important. In the first, the active ecosystem had never offered the required integration capability. In the second, finance capability existed in the ecosystem but its provider failed. Both situations trigger organizational formation, yet the reasons are different. A serious adaptive system should be able to distinguish absence of capability from loss of capacity.

The third disturbance blocks the target most recently recommended after the finance-provider failure. The company is added to the blocked set and the revision increments again. The next mission recomputes its shortlist under the new state. The final recommendation is required to differ from the excluded target. This is adaptation to a changed opportunity set.

Knowledge freshness is tested naturally across these revisions. The first repeated mission at revision zero reuses three reports and has zero new contract cost. Once the environment changes, old knowledge remains on the board but no longer matches the new revision key. Every later mission must therefore obtain fresh evidence. This is a particularly clean demonstration of why memory should be state-aware.

The summary table produced by Unit 8 records mission label, revision, provider combination, synthetic cost, number of reused reports, and selected target. It gives the learner a compact trajectory of institutional change. The organization is not simply generating different answers; it is changing which providers participate and why.

Pedagogical interpretation. Adaptation is not “the model changed its mind.” In this notebook, adaptation means that a changed environment modifies validity, discovery, organizational membership, contract formation, and the evidence that is permitted to reach the final decision.

Code Unit 9: Verify Lifecycle, Recovery, Freshness, and Transactional Cleanup

Code Unit 9 converts the notebook's architectural claims into executable tests. This is where the narrative is forced to meet the implementation. A system that claims to have memory, recovery, negotiation, dissolution, and bounded resources should be able to demonstrate each property through observable state.

The first assertions confirm the intended reuse behavior. The second mission at the original revision must reuse three reports and incur zero contract cost. Every mission after the first environmental revision must reuse zero reports. Together, these checks establish both the positive and negative sides of memory: knowledge is reused when valid and refused when stale.

The next tests verify organizational adaptation. The audit trail must contain a `provider_formed` event for integration. The mission following finance-desk withdrawal must include a report produced by `finance_backup`. The final recommendation after target withdrawal must not equal the blocked target. These are strong checks because they examine the actual realized trajectory rather than merely confirming that helper functions exist.

Lifecycle cleanup is then tested. No agents may remain in `BUSY_AGENTS`. Every alliance must be dissolved. Every successful contract must be completed. Mission cost must remain inside the base team budget. The message bus must contain the expected protocol kinds: RFP, BID, COUNTER, ACCEPT, DELIVER, and SHARE. The system is therefore validated at the levels of resources, organizations, commitments, communication, and budget.

Negative tests are especially instructive. Delivering the same completed contract twice must fail. Integrating reports from the original revision into the expanded mandate must fail because the evidence is stale or incomplete. A message from an unknown sender must fail. These tests show that governance is not only about the happy path. A mature system defines invalid transitions as carefully as valid ones.

The most important failure experiment is transactional cleanup under insufficient budget. The notebook snapshots the busy-agent set and then attempts to run a mission with a team budget of one. The mission is expected to fail. After failure, the set of busy agents must equal the snapshot, no active temporary providers may remain, and no contract may remain reserved. This is an explicit rollback property. A failed organizational attempt should not poison the institution's future capacity.

Finally, the source hashes are recomputed and compared with the original fingerprints, confirming that the shared dataset remained unchanged throughout the run. The unit then produces two diagnostic charts: teaching cost by mission and observed protocol-message counts. These plots are measurements of the controlled implementation. The notebook is careful not to describe them as evidence of real-market economic efficiency.

The output plot is valuable because it reveals the cost of reuse directly. The repeated mission at the same revision has zero new contract cost, whereas later missions require new commitments after revisions invalidate prior memory. The message chart shows the volume of RFPs, bids, counteroffers, acceptances, deliveries, shares, and rejections produced by the protocol.

Code Unit 10: Persist the Ecosystem Record and Complete the Pedagogical Ladder

The final code unit turns an in-memory experiment into an institutional record. A unique run identifier is created from a UTC timestamp and random suffix, and a dedicated output directory is created under `OUTPUTS/NB05`. The notebook then serializes the objects required to reconstruct what happened.

The saved artifacts are deliberately granular. `results.json` contains mission outcomes and analytical contributions. `messages.json` contains the protocol bus. `contracts.json` contains commitments and their final states. `knowledge_board.json` contains reusable evidence and provenance. `alliances.json` preserves mission organizations and outcomes. `directory.json` captures the final organizational structure. `audit.json` records formation, disturbances, dissolution, tool calls, and decisions. `usage.json` records model-call and token information. A trajectory CSV summarizes the sequence of missions.

The manifest is the anchor that connects these artifacts to their origins. It records a schema version, model, mode, data hashes, NB04 handoff path, and NB04 checksum. This means a future reader can establish which data, prior capability catalog, and execution configuration produced the run. Repeated execution creates a new run directory rather than overwriting the old one.

This design reflects an important shift from “answer generation” to “institutional computation.” A conventional chatbot response often disappears into conversational history. Notebook 05 instead treats the full run as an object that can be inspected after the session ends. One can select a task identifier in `messages.json`, find the associated contract, identify the specialist and provider, inspect the delivered report, locate the corresponding knowledge-board entry, and trace how that contribution entered an alliance and final decision.

The final printed sentence is deliberately restrictive: the ecosystem coordinates research; it never executes an acquisition. This is more than a disclaimer. It defines the institutional boundary of the notebook. The system can screen candidates, allocate analytical work, synthesize research, adapt its organization, and produce a research recommendation. It cannot turn that recommendation into a real transaction. The distinction between research authority and transaction authority is preserved through the entire architecture.

From the perspective of the five-step tutorial, Unit 10 completes the ladder. The deterministic tools and skills of the primitive layer remain visible inside specialist reports. The constellation layer remains visible in desk membership and coordination. Feedback and memory remain visible in the knowledge board and revision-aware reuse. Self-adaptation remains visible in temporary provider formation and replacement. Ecosystem coordination becomes visible in inter-constellation contracts, messages, alliances, and reorganization. The final artifact set preserves evidence of every layer.

This cumulative construction is pedagogically stronger than presenting an autonomous agent as a monolith. The learner can point to the exact mechanism that creates each form of agency. Autonomy emerges from composition: bounded reasoning, deterministic tools, permissions, state, protocols, memory, resource constraints, validation, and audit.

Conclusion: From Model Intelligence to Institutional Intelligence

Notebook 05 is most valuable when it is read as a model of institutional intelligence rather than as a demonstration of many agents talking to one another. Its central achievement is to move the locus of adaptability from one model to an organization of bounded components. Individual specialists remain narrow. Tools remain deterministic. Capabilities remain approved. What becomes dynamic is the arrangement through which those components are discovered, contracted, reused, replaced, and retired.

This shift matters because real financial work is distributed. A serious M&A process does not rely on one undifferentiated intelligence. Financial analysis, strategic assessment, evidence review, cross-border questions, integration risk, legal analysis, financing, tax, and execution belong to different domains and often to different teams. Notebook 05 abstracts that reality into a tractable laboratory. It does not pretend to reproduce an investment bank. It isolates a small set of organizational mechanisms that can be understood in code.

The first mechanism is capability separation. The system knows that finance is not strategy, that intelligence is not legal clearance, and that integration complexity is not a synergy estimate. Those boundaries are encoded in skills, tools, and permissions. The second mechanism is institutional structure. Approved specialists exist in a registry, but active providers exist in a directory. This creates the possibility that capabilities can be organizationally absent even when the underlying expertise exists. The third mechanism is contracting. Discovery becomes a commitment only after bounded negotiation, budget checking, and capacity reservation. The fourth mechanism is provenance-aware delivery. Work is valuable because the institution can reconstruct who produced it, under which task, against which evidence, under which revision, and for which mandate.

The fifth mechanism is governed memory. The notebook’s knowledge board shows why memory should not be equated with unrestricted retrieval. A report is reusable only when the capability, buyer, candidate scope, and revision all

match. When the environment changes, the old report does not disappear. It becomes historical evidence rather than current evidence. This is a subtle but powerful distinction. A financial institution must often remember old analysis precisely so that it can explain why it is no longer relying on it.

The sixth mechanism is organizational formation. When the expanded mandate requires integration analysis, the ecosystem does not ask an existing strategy analyst to improvise an unapproved skill. It searches the registry for a qualified, available, unallocated integration specialist and creates a temporary provider around that approved primitive. When the finance desk later withdraws, the system does not pretend that finance is optional. It finds the approved backup analyst and forms a replacement provider. Adaptation therefore occurs through reorganization of governed components.

The seventh mechanism is revision. Environmental change is not represented merely as a new prompt. It becomes a new system state. Outstanding work is cancelled before the revision changes. New missions are planned under the new revision. Old knowledge no longer matches the freshness key. The audit log records the event. This makes the boundary between before and after explicit.

The eighth mechanism is transactional cleanup. The failed-budget test is especially important because it demonstrates that the ecosystem is not allowed to leave partial organizational state after a failed mission. Reserved specialists are released. Temporary providers are retired. Pending contracts do not survive. A system that can adapt but cannot clean up failed adaptations will eventually become incoherent.

The optional use of `gpt-5-nano` should be understood in this context. The language model supplies interpretation and bounded selection at specific points. It does not define the institution. Deterministic code still controls who exists, what tools are authorized, what counts as current evidence, which messages are valid, how a contract becomes binding, when capacity is reserved, when a report becomes stale, and what artifacts must be persisted. This separation is one of the notebook's strongest lessons for practitioners. Model intelligence is powerful, but institutional reliability is architectural.

The notebook also remains appropriately modest about emergence. Provider combinations and replacement arrangements are computed during execution; they are not all hard-coded in advance. Yet the permissible interaction rules, disturbance types, budget rules, and formation mechanisms are programmed. The system therefore demonstrates bounded adaptive coordination, not spontaneous general intelligence or a decentralized economic equilibrium. That precision strengthens the educational value of the experiment.

The broader research literature provides useful parallels. Multi-agent systems have long studied distributed action, local knowledge, coordination, and negotiation (Wooldridge, 2009; Shoham and Leyton-Brown, 2009). The Contract Net Protocol formalized an early mechanism for allocating work through explicit requests and offers (Smith, 1980). More recent LLM-agent systems explore structured roles, tool use, reflective memory, and dynamic group interaction (Yao et al., 2023; Shinn et al., 2023; Wu et al., 2024; Hong et al., 2024; Chen et al., 2024). Notebook 05 contributes a pedagogical synthesis of these ideas inside a financial-research setting where governance and provenance are visible in every stage.

The final conceptual move is therefore from model intelligence to institutional intelligence. A model can reason. An institution must decide who is allowed to reason about what, with which evidence, under which mandate, using which tools, at what cost, under which state, with what memory, and with what record after the fact. Notebook 05 encodes those questions as executable objects. That is why it is a fitting capstone to the tutorial.

Implementation Appendix: What Is Pedagogical, What Is Live, and What Would Change in Production

The notebook is intentionally standalone. All constellations, coordinators, contracts, messages, knowledge entries, and disturbances exist inside one Python process. The message bus is a list rather than a distributed broker. Contracts are dictionaries rather than records in a transactional database. Specialist reservation is represented by a Python set. The directory is a dictionary. These simplifications are not presented as production infrastructure. They isolate the architectural ideas so the learner can inspect every transition.

The synthetic data layer serves the same purpose. The notebook operates on 500 synthetic companies with structured financial and strategic data plus 1,500 synthetic documents. The shared CSV files are read-only. The notebook verifies their identity before execution and preserves their hashes in the final manifest. In a production research system, these files could be replaced by market-data platforms, financial databases, document stores, diligence data rooms, CRM systems, and internal research repositories. The architectural requirement would remain the same: evidence must be versioned, attributable, and available through interfaces that preserve information boundaries.

The LIVE path is also explicitly bounded. The notebook uses the OpenAI Responses API through the Python SDK. Specialist reasoning, deal-lead synthesis, and temporary-provider selection may call `gpt-5-nano`. Every model output is constrained by a JSON schema. The request cache includes the current revision so that a response generated under one state is not silently reused under another. The notebook limits total requests and output tokens and records token usage. Incomplete or refused responses are not silently replaced by simulation output. The run stops

so the operator can inspect the failure. This is an important design choice: deterministic simulation and live model execution are two explicit modes, not interchangeable sources of answers.

A production extension would need substantially stronger infrastructure. Identity and authorization would need to be externalized. Message delivery would require durable ordering and idempotency guarantees. Contract reservation would need transactional storage. Concurrent missions could create races for specialist capacity that this sequential notebook does not simulate. Failure recovery would need to address partial network failure, process restarts, duplicate messages, delayed delivery, and distributed locks. The local invariants demonstrated here would still be useful, but they would need to be enforced across services rather than within one Python interpreter.

Likewise, real M&A usage would require stronger professional controls. Financial models would require validation and version governance. Legal, tax, regulatory, financing, and compliance work would need appropriately qualified human or controlled-system ownership. The research-only status would need to be enforced through identity, permissions, workflow gates, and documentation. The notebook should therefore be understood as an architectural laboratory, not as a production due-diligence or transaction-execution system.

The consolidated-code output embedded in the notebook reproduces the same teaching diagnostics as Unit 9. The notebook's individual units and its consolidated executable form lead to the same observable class of metrics. The slight differences in exact message counts reflect the executed run state, not a change in architectural meaning.

References

References

- Alejandro Reynoso. *AI Finance: From Agents to Ecosystems—Notebooks and Companion Research Papers*. Public GitHub repository, 2026. https://github.com/alexdiabol/ai_finance_from_agents_to_ecosystems/tree/main/notebooks.
- Tutorial of Agentic Systems. *Notebook 05 – Build Dynamic Agentic Ecosystems: Constellations, Discovery, Agreements, Shared Knowledge, and Adaptation*. Topic 766 pedagogical notebook, 2026.
- Reid G. Smith. The Contract Net Protocol: High-Level Communication and Control in a Distributed Problem Solver. *IEEE Transactions on Computers*, C-29(12):1104–1113, 1980.
- Michael Wooldridge. *An Introduction to MultiAgent Systems*. John Wiley & Sons, 2nd edition, 2009.
- Yoav Shoham and Kevin Leyton-Brown. *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Cambridge University Press, 2009.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*, 2023.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. In *Conference on Language Modeling*, 2024.
- Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiewu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework. In *International Conference on Learning Representations*, 2024.
- Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, Yujia Qin, Xin Cong, Ruobing Xie, Zhiyuan Liu, Maosong Sun, and Jie Zhou. AgentVerse: Facilitating Multi-Agent Collaboration and Exploring Emergent Behaviors. In *International Conference on Learning Representations*, 2024.